UNIVERSIDADE ESTÁCIO DE SÁ

CAMPUS MADUREIRA

Técnico de Informática

NATHAN FELIPE TUNÃO CERQUEIRA LOUREIRO

PROJETO DE BANCO DE DADOS: LOJA VIRTUAL

Modelagem, Implementação e Documentação

Rio de Janeiro

2026

Sumário

1 - Apresentação do Projeto Objetivos ---------- 3

2 - Temas e Cenários ---------- 3

3 Fase 1: Planejamento e Design ----------3

4 Fase 2: Modelo Lógico e Normalização ---------- 7

5 Fase 3: Modelo Físico e Implementação ---------- 8

6 Fase 4: Consultas de Validação ---------- 13

7 Fase 4: Testes de Integridade ---------- 14

11 Relatório final ---------- 15

Descrição do Projeto Loja Virtual

O Projeto trata do desenvolvimento de um sistema de banco de dados para uma loja virtual que gerenciará o fluxo de vendas, controlando o cadastro de clientes, o estoque de produtos, os pedidos de compra com seus respectivos itens e as formas de pagamento utilizadas e o gerenciamento dos fornecedores.

O cronograma foi divido em 5 fases que incluíram o planejamento, modelagem (conceitual, lógico e físico), implementação, teste e validação.

Levantamento de requisitos

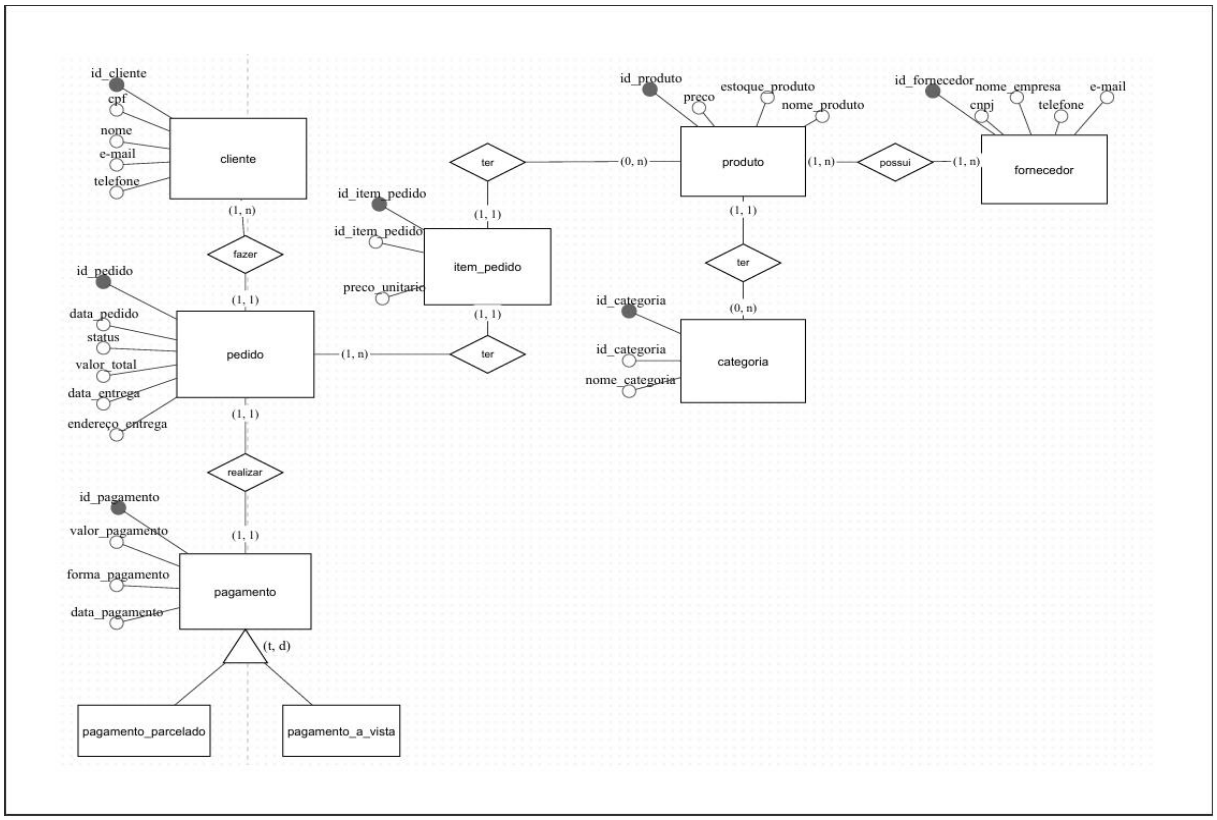
- Gerenciar o cadastro de clientes;
- Gerenciar o catálogo de produtos e suas categorias;
- Cadastrar fornecedores;
- Processar pedidos e seus respectivos itens;
- Registrar os pagamentos e suas respectivas formas.

Regras de negócio

- Todo pedido deverá estar, obrigatoriamente, vinculado a um cliente;
- O pedido poderá ter 1 (um) ou vários produtos;
- Cada produto deverá estar, obrigatoriamente, associado a uma categoria;
- O pedido deverá ser, obrigatoriamente, registrado somente como uma forma de pagamento, parcelado ou a vista;
- O produto poderá ter muito fornecedores, assim como os fornecedores poderão fornecer muitos produtos.

Modelo Entidade Relacionamento (MER)

A figura a seguir apresenta o modelo conceitual do banco de dados.



Dicionário Conceitual

Entidade cliente			
Nome do Campo	Tipo de dado	Descrição	Restrições
Id_cliente	INT	Identificação do cliente	Primary key, Not Null e Auto Increment
cpf	VARCHAR	Documento do cliente	Not Null e Unique
nome	VARCHAR	Nome do cliente	Not Null
e-mail	VARCHAR	Endereço eletrônico para contato	Not Null e Unique
telefone	VARCHAR	Contato para comunicação direta e rápida.	Not Null e Unique

Entidade pedido			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_pedido	INT	Identificação do Pedido.	Primary key, Not Null e Auto Increment
id_cliente	INT	Relação entre as entidades cliente e pedido.	Not Null
data_pedido	DATE	Data em que o pedido foi feito.	Not Null
status_pedido	VARCHAR	Situação do pedido.	Not Null
valor_total	DECIMAL	Valor total de todos os produtos.	Not Null
data_entrega	DATE	Data prevista para entrega do pedido.	Null (opcional).
endereço_entrega	VARCHAR	Endereço em que será entregue o pedido.	Not Null

Entidade pagamento_parcelado			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_pagamento	INT	Identificador e relação entre as entidades pagamento e pagamento_parcelado.	Primary key e Not Null.
num_parcelas	INT	Quantidades de parcelas do pagamento.	Not Null.

Entidade pagamento			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_pagamento	INT	Identificador do pagamento.	Primary key, Not Null e Auto Increment
id_pedido	INT	Relação entre as entidades pedido e pagamento.	Not Null
valor_pagamento	DECIMAL	Valor do pedido a ser pago.	Not Null
forma_pagamento	VARCHAR	Tipo de pagamento	Not Null
data_pagamento	DATE	Data em que foi pago.	Not Null

Entidade pagamento_a_vista			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_pagamento	INT	Identificador e relação entre as entidades pagamento e pagamento_a_vista.	Primary key e Not Null.
tipo_pagamento	VARCHAR	Forma de pagamento a vista.	Not Null.
desconto	DECIMAL	Desconto no pagamento.	Null.

Entidade item_pedido			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_item_pedido	INT	Identificador do pedido.	Primary key, Not Null e Auto Increment.
id_pedido	INT	Relação entre as entidades pedido e item_pedido.	Not Null
id_produto	INT	Relação entre as entidades produto e item_pedido.	Null (opcional).
quantidade	INT	Quantidade de produtos.	Not Null
preco_unitario	DECIMAL	Valor unitário de cada produto	Not Null

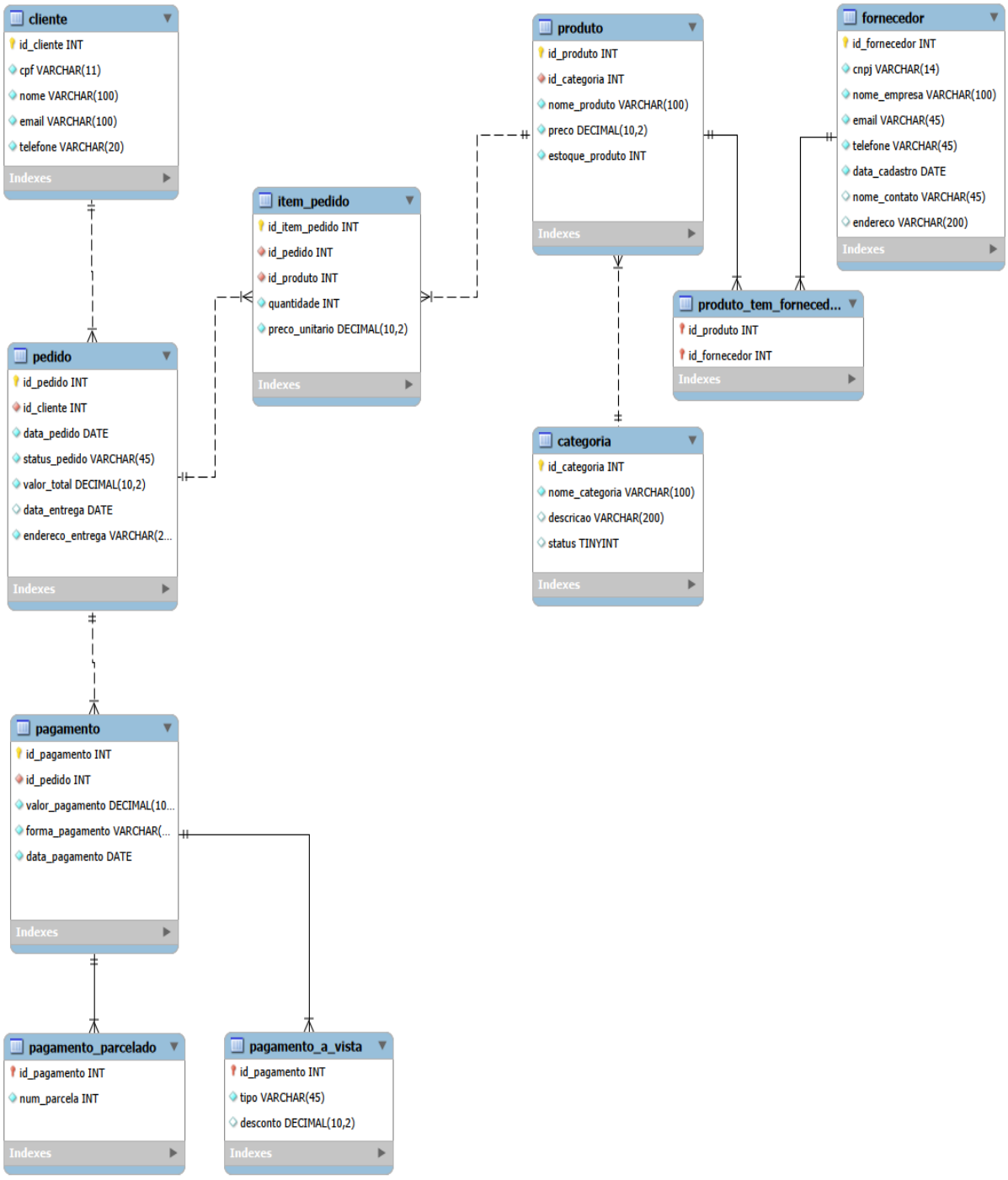
Entidade produto			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_produto	INT	Identificador do produto.	Primary key, Not Null e Auto Increment.
id_categoria	INT	Relação entre as entidades produto e categoria.	Not Null
nome_produto	VARCHAR	Nome do produto	Not Null
preco	DECIMAL	Valor do produto	Not Null
estoque_produto	INT	Quantidade do produto no estoque	Not Null

Entidade categoria			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_categoria	INT	Identificador da categoria.	Primary key, Not Null e Auto Increment.
nome_categoria	VARCHAR	Nome da categoria.	Not Null.
descricao	VARCHAR	Descrição de que tipo de produtos a categoria terá.	Null.
status	TINNYINT	Situação da categoria.	Null.

Entidade fornecedor			
Nome do Campo	Tipo de dado	Descrição	Restrições
id_fornecedor	INT	Identificador do fornecedor.	Primary key, Not Null e Auto Increment.
cnpj	VARCHAR	Documento do fornecedor.	Not Null.
nome_empresa	VARCHAR	Nome da empresa.	Not Null.
email	VARCHAR	Endereço eletrônico para contato.	Not Null e Unique.
telefone	VARCHAR	Contato para comunicação direta e rápida.	Not Null e Unique.
nome_contato	VARCHAR	Nome para entrar em contato.	Null.
endereco	VARCHAR	Endereço da empresa.	Null.
data_cadastro	DATE	Data em que a empresa foi cadastrada.	Not Null.

Modelo Lógico

A figura a seguir apresenta o modelo lógico do banco de dados, normalizado até a 3ª forma normal (3FN), ilustrando a estrutura relacional das tabelas, suas chaves primárias e estrangeiras.



Modelo Físico e Implementação

Este script inicia a estrutura do banco de dados para a loja virtual.

```
-- -----------------------------------------------------
-- Schema projeto_loja_virtual
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS `projeto_loja_virtual` DEFAULT
CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci ;
USE `projeto_loja_virtual` ;
```

A tabela categoria armazenará informações básicas sobre determinado produto e está vinculada a tabela produto.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`categoria`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`categoria`
(
  `id_categoria` INT NOT NULL AUTO_INCREMENT,
  `nome_categoria` VARCHAR(100) NOT NULL,
  `descricao` VARCHAR(200) NULL DEFAULT NULL,
  `status` TINYINT(1) NULL DEFAULT NULL,
  PRIMARY KEY (`id_categoria`))
ENGINE = InnoDB
AUTO_INCREMENT = 6
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela cliente conterá todas as informações referentes ao cliente.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`cliente`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`cliente` (
  `id_cliente` INT NOT NULL AUTO_INCREMENT,
  `cpf` VARCHAR(11) NOT NULL,
  `nome` VARCHAR(100) NOT NULL,
  `email` VARCHAR(100) NOT NULL,
  `telefone` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`id_cliente`),
  UNIQUE INDEX `E-mail_UNIQUE` (`email` ASC) VISIBLE,
  UNIQUE INDEX `telefone_UNIQUE` (`telefone` ASC) VISIBLE,
  UNIQUE INDEX `cpf_UNIQUE` (`cpf` ASC) VISIBLE,
  INDEX `idx_nome_cliente` (`nome` ASC) VISIBLE)
ENGINE = InnoDB
AUTO_INCREMENT = 13
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela Fornecedor conterá todas as informações referentes ao fornecedor.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`fornecedor`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`fornecedor`
(
  `id_fornecedor` INT NOT NULL AUTO_INCREMENT,
  `cnpj` VARCHAR(14) NOT NULL,
  `nome_empresa` VARCHAR(100) NOT NULL,
  `email` VARCHAR(45) NOT NULL,
  `telefone` VARCHAR(45) NOT NULL,
  `data_cadastro` DATE NOT NULL,
  `nome_contato` VARCHAR(45) NULL DEFAULT NULL,
  `endereco` VARCHAR(200) NULL DEFAULT NULL,
  PRIMARY KEY (`id_fornecedor`),
  UNIQUE INDEX `cnpj_UNIQUE` (`cnpj` ASC) VISIBLE,
  UNIQUE INDEX `telefone_UNIQUE` (`telefone` ASC) VISIBLE,
  UNIQUE INDEX `e-mail_UNIQUE` (`email` ASC) VISIBLE)
ENGINE = InnoDB
AUTO_INCREMENT = 11
DEFAULT CHARACTER SET = utf8mb3
COMMENT = '            ';
```

A tabela pedido guardará os dados do pedido, vinculando-o às tabelas cliente, pagamento e item_solicitado.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`pedido`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`pedido` (
  `id_pedido` INT NOT NULL AUTO_INCREMENT,
  `id_cliente` INT NOT NULL,
  `data_pedido` DATE NOT NULL,
  `status_pedido` VARCHAR(45) NOT NULL,
  `valor_total` DECIMAL(10,2) NOT NULL,
  `endereco_entrega` VARCHAR(200) NOT NULL,
  `data_entrega` DATE NULL DEFAULT NULL,
  PRIMARY KEY (`id_pedido`, `id_cliente`),
  INDEX `fk_pedido_cliente1_idx` (`id_cliente` ASC) VISIBLE,
  INDEX `idx_id_cliente` (`id_cliente` ASC) VISIBLE,
  CONSTRAINT `fk_pedido_cliente1`
    FOREIGN KEY (`id_cliente`)
    REFERENCES `projeto_loja_virtual`.`cliente`
(`id_cliente`))
ENGINE = InnoDB
AUTO_INCREMENT = 8
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela de produtos guardará dados referentes ao produto, estabelecendo uma relação com a tabela de categorias.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`produto`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`produto` (
  `id_produto` INT NOT NULL AUTO_INCREMENT,
  `id_categoria` INT NOT NULL,
  `nome_produto` VARCHAR(100) NOT NULL,
  `preco` DECIMAL(10,2) NOT NULL,
  `estoque_produto` INT NOT NULL,
  PRIMARY KEY (`id_produto`),
  INDEX `fk_produto_categoria_idx` (`id_categoria` ASC)
VISIBLE,
  CONSTRAINT `fk_produto_categoria`
    FOREIGN KEY (`id_categoria`)
    REFERENCES `projeto_loja_virtual`.`categoria`
(`id_categoria`))
ENGINE = InnoDB
AUTO_INCREMENT = 11
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela item_pedido registrará dados a respeito dos itens solicitados, estabelecendo uma conexão entre as tabelas pedido e produto.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`item_pedido`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS
`projeto_loja_virtual`.`item_pedido` (
  `id_item_pedido` INT NOT NULL AUTO_INCREMENT,
  `id_pedido` INT NOT NULL,
  `id_produto` INT NOT NULL,
  `quantidade` INT NOT NULL,
  `preco_unitario` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`id_item_pedido`),
  INDEX `fk_item_pedido_pedido_idx` (`id_pedido` ASC) VISIBLE,
  INDEX `fk_item_pedido_produto_idx` (`id_produto` ASC)
VISIBLE,
  CONSTRAINT `fk_item_pedido_pedido`
    FOREIGN KEY (`id_pedido`)
    REFERENCES `projeto_loja_virtual`.`pedido` (`id_pedido`),
  CONSTRAINT `fk_item_pedido_produto`
    FOREIGN KEY (`id_produto`)
    REFERENCES `projeto_loja_virtual`.`produto`
(`id_produto`))
ENGINE = InnoDB
AUTO_INCREMENT = 12
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela pagamento registrará dados do pagamento e está ligada diretamente a tabela peido.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`pagamento`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `projeto_loja_virtual`.`pagamento`
(
  `id_pagamento` INT NOT NULL AUTO_INCREMENT,
  `id_pedido` INT NOT NULL,
  `valor_pagamento` DECIMAL(10,2) NOT NULL,
  `forma_pagamento` VARCHAR(30) NOT NULL,
  `data_pagamento` DATE NOT NULL,
  PRIMARY KEY (`id_pagamento`),
  INDEX `fk_pagamento_pedido1_idx` (`id_pedido` ASC) VISIBLE,
  CONSTRAINT `fk_pagamento_pedido1`
    FOREIGN KEY (`id_pedido`)
    REFERENCES `projeto_loja_virtual`.`pedido` (`id_pedido`))
ENGINE = InnoDB
AUTO_INCREMENT = 5
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela pagamento_a_vista registrará dados do pagamento a vista e está associada a tabela pagamento.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`pagamento_a_vista`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS
`projeto_loja_virtual`.`pagamento_a_vista` (
  `id_pagamento` INT NOT NULL,
  `tipo` VARCHAR(45) NOT NULL,
  `desconto` DECIMAL(10,2) NULL DEFAULT NULL,
  PRIMARY KEY (`id_pagamento`),
  CONSTRAINT `fk_pag_a_vista_pagamento`
    FOREIGN KEY (`id_pagamento`)
    REFERENCES `projeto_loja_virtual`.`pagamento`
(`id_pagamento`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela pagamento_parcelado registrará dados do pagamento parcelado e está associada a tabela pagamento.

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`pagamento_parcelado`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS
`projeto_loja_virtual`.`pagamento_parcelado` (
  `id_pagamento` INT NOT NULL,
  `num_parcela` INT NOT NULL,
```

```
  PRIMARY KEY (`id_pagamento`),
  CONSTRAINT `fk_pag_parcelado_pagamento`
    FOREIGN KEY (`id_pagamento`)
    REFERENCES `projeto_loja_virtual`.`pagamento`
(`id_pagamento`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb3;
```

A tabela produto_tem_fornecedor, conecta às tabelas produto e fornecedor, utilizando o relacionamento muitos para muitos (N:N) .

```
-- -----------------------------------------------------
-- Table `projeto_loja_virtual`.`produto_tem_fornecedor`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS
`projeto_loja_virtual`.`produto_tem_fornecedor` (
  `id_produto` INT NOT NULL,
  `id_fornecedor` INT NOT NULL,
  PRIMARY KEY (`id_produto`, `id_fornecedor`),
  INDEX `fk_produto_has_fornecedor_fornecedor1_idx`
(`id_fornecedor` ASC) VISIBLE,
  INDEX `fk_produto_has_fornecedor_produto1_idx` (`id_produto`
ASC) VISIBLE,
  CONSTRAINT `fk_produto_has_fornecedor_fornecedor1`
    FOREIGN KEY (`id_fornecedor`)
    REFERENCES `projeto_loja_virtual`.`fornecedor`
(`id_fornecedor`),
  CONSTRAINT `fk_produto_has_fornecedor_produto1`
    FOREIGN KEY (`id_produto`)
    REFERENCES `projeto_loja_virtual`.`produto`
(`id_produto`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb3;


USE `projeto_loja_virtual` ;
```

Esta Views será utilizada para uma consulta virtual que facilitará a visualização dos dados combinados entre as tabelas pedidos e pagamentos.

```
CREATE TABLE IF NOT EXISTS
`projeto_loja_virtual`.`view_pedidos_pagamentos` (`id_pedido`
INT, `id_cliente` INT, `data_pedido` INT, `status_pedido` INT,
`valor_total` INT, `endereco_entrega` INT, `data_entrega` INT,
`valor_pagamento` INT, `forma_pagamento` INT, `data_pagamento`
INT);
```

Esta Views será utilizada para uma consulta virtual que facilitará a visualização dos dados combinados entre as tabelas pedidos e clientes.

```
create view view_pedido_pagamento as select p.id_pedido,
```

```
p.data_pedido, p.valor_total, pg.valor_pagamento,
pg.forma_pagamento, pg.data_pagamento from pedido p join
pagamento pg on p.id_pedido = pg.id_pedido;
select * from view_pedido_pagamento;
```

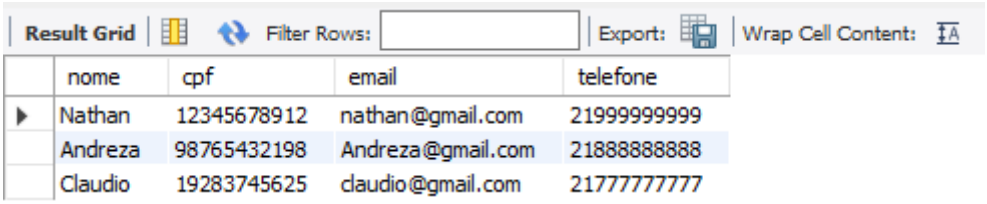
O modelo físico foi implementado com sucesso, criando as tabelas e views do banco de dados.

Consultas e Validação

Os testes de consulta de validação visam confirmar a integridade e o funcionamentos das tabelas e views. Abaixo, visualizaremos alguns códigos que foram utilizados para conferir a integridade do banco de dados.

O código de consulta **SELECT** abaixo, apresentará todas as informações, conforme os campos descritos, da tabela cliente, seguido da imagem com o resultado.

```
select nome, cpf, email, telefone from cliente;
```



nome	cpf	email	telefone
Nathan	12345678912	nathan@gmail.com	21999999999
Andreza	98765432198	Andreza@gmail.com	21888888888
Claudio	19283745625	claudio@gmail.com	21777777777

O código abaixo, fará uma consulta do nome e preço dos produtos e em seguida fará uma **SUBCONSULTA** para identificar quais produtos estão acima da média.

```
select nome_produto, preco from produto where preco > (select
avg (preco) from produto);
```



nome_produto	preco
Redmi note 9s	1280.59
Notbook	1999.99

O código de **consulta** abaixo, **validará** e apresentará todas as informações, conforme os campos descritos, entre a tabela cliente e tabela pedidos, seguido da imagem com o resultado.

```
select p.id_pedido from pedido p left join cliente c on
p.id_cliente = c.id_cliente where c.id_cliente is null;
```

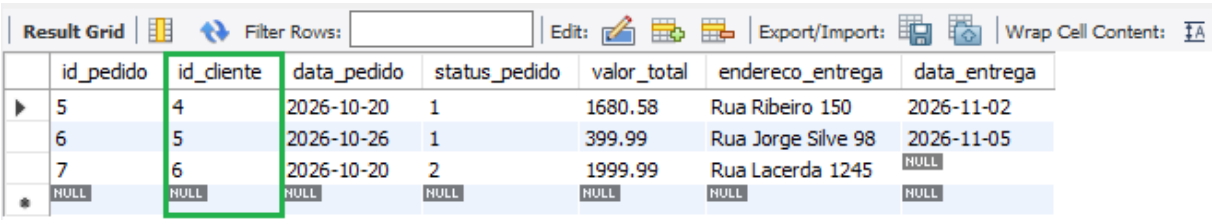


id_pedido

Este teste indica que não existe nenhum pedido sem registro de um cliente, comprovando que todo pedido precisa estar relacionado a um cliente. Para confirmar o teste, utilizamos o código

select * from “nome da tebela”, para conferir o relacionamento entre elas, conforme o código e a imagem abaixo.

```
Select * from peido;
```



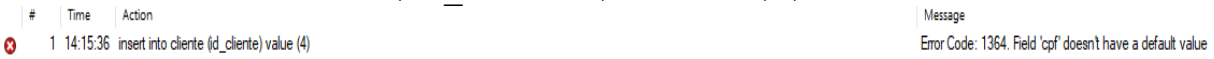
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	id_cliente	data_pedido	status_pedido	valor_total	endereco_entrega	data_entrega
5	4	2026-10-20	1	1680.58	Rua Ribeiro 150	2026-11-02
6	5	2026-10-26	1	399.99	Rua Jorge Silve 98	2026-11-05
7	6	2026-10-20	2	1999.99	Rua Lacerda 1245	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Teste de Integridade

Ignorando os campos Not Null:

```
insert into cliente (id_cliente) value (4);
```



#	Time	Action	Message
1	14:15:36	insert into cliente (id_cliente) value (4)	Error Code: 1364. Field 'cpf' doesn't have a default value

Erro: exigiu o campo **CPF.**

```
insert into cliente (id_cliente, cpf) value (4, 12345678912);
```



2	14:20:50	insert into cliente (id_cliente, cpf) values (4, '12345678912')	Error Code: 1364. Field 'nome' doesn't have a default value

Erro: exigiu o campo **Nome.**

insert into cliente (id_cliente, cpf, nome) values (4, '12345678912','Nathan');



3	14:22:35	insert into cliente (id_cliente, cpf, nome) values (4, '12345678912','Nathan')	Error Code: 1364. Field 'email' doesn't have a default value

Erro: exigiu o campo **Email.**

```
insert into cliente (id_cliente, cpf, nome, email) values (4,
'12345678912','Nathan','nathan@gmail.com');
```

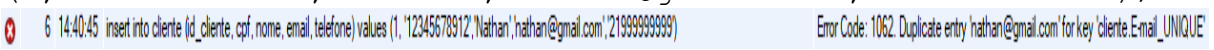


4	14:26:57	insert into cliente (id_cliente, cpf, nome, email) values (4, '12345678912','Nathan','nathan@gmail.com')	Error Code: 1364. Field 'telefone' doesn't have a default value

Erro: exigiu o campo **Telefone.**

Duplicando os campos **Unique**:

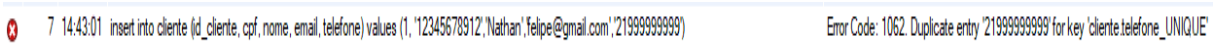
```
insert into cliente (id_cliente, cpf, nome, email, telefone)
values
(1,'12345678912','Nathan','nathan@gmail.com','21999999999');
```



6	14:40:45	insert into cliente (id_cliente, cpf, nome, email, telefone) values (1, '12345678912','Nathan','nathan@gmail.com','21999999999')	Error Code: 1062. Duplicate entry 'nathan@gmail.com' for key 'cliente.E-mail_UNIQUE'

Erro: Informou duplicidade do campo **e-mail**.

```
insert into cliente (id_cliente, cpf, nome, email, telefone)
values
(1,'12345678912','Nathan','felipe@gmail.com','21999999999');
```



Erro: Informou duplicidade do campo **telefone**.

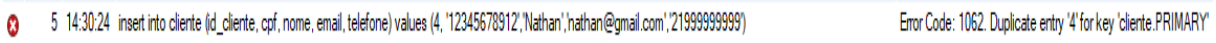
```
insert into cliente (id_cliente, cpf, nome, email, telefone)
values
(1,'12345678912','Nathan','felipe@gmail.com','21555555555');
```



Erro: Informou duplicidade do campo **CPF**.

Violando a **Pimary Key**:

```
insert into cliente (id_cliente, cpf, nome, email, telefone)
values
(4,'12345678912','Nathan','nathan@gmail.com','21999999999');
```



Erro: Informou duplicidade da chave **primária**.

Violando a **Foreign key**:

insert into produto (nome_produto, preco, estoque_produto) values ("Carregador Portátil", 159.99, 50);



Erro: Informou que não foi informado a chave estrangeira “id_categoria”.

Relatório final

O projeto para a criação de um banco de dados para uma loja virtual estruturou-se através da criação de tabelas, chaves e regras de negócio, consolidando a arquitetura completa para garantir a integridade e o funcionamento esperado do sistema.
O banco de dados foi armazenado com sucesso na nuvem, o que simplifica o acesso a ele em qualquer local, desde que haja uma conexão com um computador que possua o programa MySQL Workbench e que todos os dados do servidor sejam inseridos de maneira correta, incluindo o CA certificate, sendo necessário o download e o direcionamento correto do local onde for salvo e alterar o SSL/TSL para “Require”.
Após a implementação no servidor da Aiven.com, todo o conteúdo do projeto foi lançado no repositório do GitHub e em seguida vinculado ao Render.com.

Segue em anexo o link contendo todos os arquivos do projeto:
https://github.com/NathanFlipe/Projeto_DEVOPS_BDD_Loja_Virtual